Mod SDK changelog

This is the changelog for the Core Keeper Mod SDK. It will contain changes to the SDK itself as well as internal changes to the project that isn't covered in the regular changelog.

It is still very much recommended to also read what is in the changelog for the Core Keeper application on Steam: https://store.steampowered.com/news/app/1621690?updates=true

1.3.0.5

- Water painted in the ModSDK's custom scene editor now uses its in-game color, making it easier to tell water types apart.
- Tiles without an icon in the ModSDK's custom scene editor now get a button preview generated from their tileset sprite sheet instead of an empty button.
- Added a "Hide Placeable Objects" toggle to the ModSDK's custom scene editor, which hides placed objects in the Scene view so only tiles are visible. This makes it easier to paint tiles in scenes with lots of objects.
- Fixed object IDs of modded objects depending on the order mods were loaded in, which could cause modded objects to get mixed up when the load order changed.
- Fixed Steam Workshop mods still being loaded when launching the game with the no mods option.
- Fixed modded tilesets using a tilesetType value outside the Tileset enum causing other tilesets to go missing.

1.3.0.3

- Increased the ghost component cap from 256 to 512.
- Made necessary runtime changes to add support for PugText previews in the ModSDK.
- Added an event, ECSManager.BeforeInitialEntityConversion, which can be used in IMod.EarlyInit or IMod.Init to add components to entities before conversion.
- Fixed an issue where an entity using ObjectID.None caused the player's character to hold that entity's representative graphical object in-game even when nothing was selected in the hotbar.
- Fixed an issue where UGC materials weren't being swapped to their runtime variants.
- Fixed an issue where adding required materials beyond the cap resulted in excessive error log spam, which lagged the game.
- Fixed graphical object data block overloads not pooling the correct graphical object.
- Fixed the revealExploredAreas console command so it works again in PC builds.

1.3.0.0

- Official Steam Workshop Support coming with 1.3.0 release! To support Steam Workshop, we've added an in-game mod list which shows all installed mods and their

statuses. The mod list can also be used by quickly navigate to your mods sources by clicking on them or to re-order the loading order of mods in-game. If a mod has failed to compile there will be a tooltip which will tell you if your mod failed to compile or was missing a dependency, if you click on that tooltip you'll immediately be taken to the Player.Log file where you can find more detailed information.

- Moddable Custom Scenes! You may now create your own, unique custom scenes and add them to the game via a mod.
- EntityAuthoringDataBlock is a new ScriptableDataBlock type which stores a soft reference to an Entity prefab. You may overload these datablock instances to mod existing entities, or create new ones. Where an entity prefab was previously referenced you will now have to create an EntityAuthoringDataBlock instance, reference your entity prefab to that instance and then reference the instance itself. (see the bottom of this change list for migration scripts!)
- GraphicalObjectDataBlock is a new ScriptableDataBlock type which stores a soft reference to a Graphical prefab, as well as pool settings for that prefab. You may overload these datablock instances to mod existing graphical objects, or create new ones. Where a graphical prefab was previously referenced you will now have to create a GraphicalObjectDataBlock instance, reference your graphical prefab to that instance and then reference the instance itself. (see the bottom of this change list for migration scripts!)
- ParticleEffectDataBlock is a new ScriptableDataBlock type which stores a soft reference to a Particle prefab, as well as pool settings for that prefab. You may overload these datablock instances to mod existing particles, or create new ones. ParticleEffectIDs are now deprecated and you should use ParticleEffectDataBlock instances in their place. PuffID is still used, to the same extent as before.
- PoolParameterDataBlock is a new ScriptableDataBlock type which stores the graphical object pooling size. You may either create new ones with unique parameters or re-use existing ones by assigning them from the Core Keeper Assets package. This is the new and supported way to pool objects. PooledObjectBank is now deprecated.
- TilesetDataBlock is a new ScriptableDataBlock type which stores references to Tileset textures and their adaptive tileset textures. To mod a tileset you would have to assign a new Tileset texture and regenerate the adaptive tileset textures at the bottom of your TilesetDataBlock instance. Note that the references on these TilesetDataBlock instances are addressable texture2D references, meaning that you'll need to create addressables settings first. It's crucial that when you do create addressables settings that you select Ignore when prompted with what to do with legacy bundles. Once you've set up addressables settings you may mark any tilesets which you want to use as addressable. You may then reference them on the TilesetDataBlock instances.
- CustomSceneDataBlock is a new ScriptableDataBlock type which is used for custom scene creation. Upon creating a new instance of this datablock type you will be able to press Create scene from template to get started with Custom Scene creation. Within that same datablock instance there is data stored of where which prefabs

should be placed and such, this data will be automatically populated when scenes are processed during mod building so only edit this data in advanced use cases.

- ObjectIDCategoryDataBlock is a new ScriptableDataBlock type which stores all of the game's Object ID Categories, these are primarily used for displaying objects categories in the editor as well as in creative mode menu to determine where which objects should appear when searching for a building block as an example.
- MapMarkerIconDataBlock is a new ScriptableDataBlock type which stores references to map marker icons.
- BehaviourTagsDataBlock is a new ScriptableDataBlock type which stores NPC behaviour tags as a list.
- CursorSkinDataBlockCollection is a new Scriptable Data Collection which stores references to Sprite Assets and Sprite Asset Skins for Cursor recolors. To add new recolors you may simply add a new instance of a Sprite Asset Skin and add to this collection.
- Example Mods have been consolidated into fewer, larger example mods. The documentation to reflect these changes and more will be uploaded to GitBook when 1.3.0 is officially released.
- Steam Workshop mods can now be uploaded to Steam Workshop without initializing the Core Keeper app via Unity. This is done by using an external app through Unity to handle the uploading process itself.
- Within the ModSDK, the Mod.IO interface has been refactored to fall more in line with the Steam Workshop interface. This has also resulted in multiple bug fixes such as sometimes not being able to upload an image or to upload the mod.
- Game versioning for mods will now happen automatically by tagging mods with the game version retrieved when updating game files. IE a mod which was made using the 1.3.0 game version will automatically be tagged with the 1.3.0 game version when being uploaded to Steam Workshop or Mod.IO.
- OverrideNetworkSyncDistanceAuthoring is deprecated in favor of OverrideClientSyncScreenDistanceAuthoring. You now also have the option to implement INetworkSyncDistanceProvider directly on an authoring component if values in that component indicate how far away entities need to be synchronized to clients.
- Added LightSourceAuthoring as a replacement for ActAsLightSourceWhenHeldInHandAuthoring, GlowLightAuthoring, and TableItemLightSourceAuthoring, all of which are now deprecated. This new component implements INetworkSyncDistanceProvider, meaning placeable light sources using this will automatically be synced correctly when in range.
- BehaviourTagsAuthoring now uses BehaviourTagsDataBlock. The previous fields in BehaviourTagsAuthoring still exist but are still available (hidden) to support upgrading existing content.

- ObjectInfo.prefabInfos has been deprecated and will be removed in a future update. Replaced with authoringRef and graphicalRef for EntityAuthoringDataBlock refs and GraphicalObjectDataBlock refs respectively.
- ObjectAuthoring.graphicalPrefab has been deprecated and graphicalRef should be used in it's place.
- ParticleEffectID has been deprecated in favor of ParticleEffectDataBlock instances.
- MapWorkshopTilesetBank has been refactored into instances of TilesetDataBlock.
- CustomScenesTable and CustomScenesDataTable have been refactored into instances of CustomSceneDataBlock.
- OnSpawn should now be used to call any functions relative to modifying an entity, instead of OnOccupied.

1.2.1.3

- When uploading a mod to Steam Workshop, the full list of tags selected will now be correctly saved to the Steam Workshop mod settings scriptable object.
- Creating descriptions for mods in the ModSDK has been made a bit easier, you can now create a description.txt file and drop it in your mods' folder which'll be parsed and used as the description for your mod when uploading it to Steam Workshop, given that the description field is left empty.
- AnimationClip and AnimatorController folders will now be imported alongside other Core Keepers assets from the asset ripper rip of the game.
- Improved mod ownership checks when updating mods on Steam Workshop.
- Fixed a bug where mod file IDs wouldn't be automatically updated into the fileID field when uploading a mod to Steam Workshop for the first time.
- Previously selected mod is now remembered in the Mod Settings tab.
- Removed spritemanifest and disabled spritemanifest generation for new mods.
- An error now pops up when you try to upload a mod to Steam Workshop which hasn't been built yet.

1.2.1.0

- CraftingAuthoring now uses a `ModdedObjectID` string when ObjectID is set to ObjectID.None, which'll be converted to an ObjectID during runtime. You may use this change to add modded items to workbenches and such.
- All Graphical Prefabs which have a component that inherits from IPoolable will now automatically be pooled. In order to adjust the size of pooling or to disable it you may create a `PooledModGraphicalObjectBank` Scriptable Object via the Create menu.
- The Sprite Render on the Mouse cursor has been upgraded to a SpriteAsset which can now be modded via the ScriptableDataBlock editor.
- QuantumConsole has now been exposed to mod developers and can be switched on by setting MenuManager.enableConsole to true. It can then be opened in-game via

ctrl+x. The object suggestor supports modded Object Names when using the spawnByName command.

1.2.0.3

- More comprehensive mod SDK documentation is now available at https://modding.corekeepergame.com/.
- UnityExtensions has moved to a new PugUnityExtensions package and Pug.UnityExtensions namespace.
- PugProperties has moved to a new PugProperties package and Pug.Properties namespace.
- TileMapLookup.GetBlockingTile and TileMapLookup.HasBlockingTile has been deprecated in favor of TileMapLookup.TryGetBlockingTile and will get removed in a future release.
- PugAutomation namespace has been renamed to Pug.Automation.
- PugElectricity namespace has been merged into Pug.Automation.
- ElectricityCD is now part of the Pug.Automation namespace.
- PugConversion has been extracted as a separate package. There are a few changes that may require you to update any existing converter code.
    - PugConversion namespace has been renamed to Pug.Conversion.
    - PugConverter and PugPostConverter have been renamed to Converter and PostConverter, respectively.
    - The ConditionsTable asset is no longer cached on each individual converter. Converters that need access to the conditions table should load and cache it themselves. See e.g. LevelEntitiesBufferConverter for an example.
- Added BurstDisabler API to selectively disable burst compiler for specific systems. This is currently limited to disabling Burst for ISystem. We are working on extending this to work with Burst-compiled jobs as well. You may see some jobs running without Burst using this API currently, but results are inconsistent.
- Core Keeper now uses ScriptableDataBlocks which are just like Scriptable Objects but hold an address synced to the assets' GUID.
- Gradient Maps are now stored as a texture array inside of GradientMapDataBlocks.
- SpriteAssets are now SpriteAssetDataBlocks.
- SpriteAssetSkins are now SpriteAssetSkinDataBlocks.
- PlayerCustomizationTable is now PlayerCustomizationTableDataBlock.
- Sprite Objects now reference SpriteAssetDataBlocks and GradientMapDataBlocks.
- Mods can now be loaded from Steam Workshop as well.
- Localization now works through the TextDataBlock system, to add or change in-game text you will need to create a new TextDataBlock instead of a Localization.csv file.
- A majority of the ModSDK has been moved from assets into a PugMod package.

1.1.2.8

- Upgraded the Unity engine version from 6000.0.58f2 to 6000.0.59f2. This was required for fix a frequent source of crashes, and it is not expected to impact

modding. You can download the new version in the Unity 6 > LTS section in the download archive: https://unity.com/releases/editor/archive. If you are having issues after upgrading, try closing the editor and delete the project's Library folder before reopening.

1.1.2.6

- Upgraded the Unity engine version from 2022.3.62f2 to 6000.0.58f2. You can download the new version in the Unity 6 > LTS section in the download archive: https://unity.com/releases/editor/archive. If you are having issues after upgrading, try closing the editor and delete the project's Library folder before reopening.
- Removed the obsolete Visual Studio Code Editor package. The Visual Studio Editor package now includes support for VS Code.
- Removed the OpenProject scripts. Instead, you should open the project using the Unity Hub available for download on Unity's website.

1.1.2.5

- Upgraded the Unity engine version from 2022.3.50f1 to2022.3.62f2. You can download the new version in the Unity 2022 > LTS section in the download archive. If you are having issues after upgrading, try closing the editor and delete the project's Library folder before reopening. This upgrade is motivated by the security vulnerability found in older versions of the Unity engine. For more information, see Unity's official announcement.
- Addressables is now included as an embedded package in the SDK.

1.1.2.0

- UnityExtensions.ExtensionMethods.RoundToMultiple(x, m) now correctly rounds x to the nearest multiple of m as the name indicates, i.e. it returns k*m for an integer k such that k*m + r = x and -m / 2 <= r < m / 2. It used to instead round to the reciprocal 1/m. This change means all existing code that calls these methods must be adjusted.
- Improved performance of AlphabeticallySortedEnumPropertyDrawer. Thank you Limoka for this suggestion!
- SpriteAssetSkin now has a project view preview like SpriteAsset already did.
- SpriteAssets now support up to three separate gradient maps. The "grayscale" textures have been generalized so that the red channel selects a color from the primary gradient map, the green from the secondary, and the blue from the tertiary. This allows separate parts of the sprite to use different color maps.
- The SpriteAsset and SpriteAssetSkin gradient maps have changed due to the above change. If you used the SpriteAsset[Skin] inspector to generate the grayscale and gradient map textures, you can update the gradient format using the "Unmap to grayscale" option.
- Improved PickStringFromEnumDrawer to allow fuzzy searching of base game object IDs. Thank you Limoka for this contribution!

- Fixed some issues when using SDK on a Linux machine. Thank you Barroit for these contributions!

1.1.0

- Removed all fields related to sprite objects in EntityMonoBehavior.reskinOptions in favor of dedicated components like SpriteSkinFromEntityAndSeason and SpriteMaterialFromEntity. These fields have been deprecated since 0.7.3 and are no longer used by any non-modded objects.
- Renamed the SpriteInstancing package and namespace to Pug.Sprite. This breaks visuals in existing mods which use SpriteObject/SpriteAsset. Rebuilding the mods in the updated Mod SDK project should fix all references automatically.
- Renamed the PugRP package and namespace to Pug.RP.
- Include all System.* assemblies when updating the SDK from game files. Previously especially System.Collections.Immutable was missing causing some errors with InteractableObject script.
- Added modified DOTS packages embedded in the SDK project (can be found under Packages/). These are still fairly close to the public package versions in what they expose publicly but there have been more and more minor changes so we are including all of them for clarity and to simplify the SDK update process.

1.0.1.11

- Integrated some QoL life changes from pull request from Limoka: https://github.com/Pugstorm/CoreKeeperModSDK/pull/7

1.0.1.7

- Updated Unity version from 2022.3.20 to 2022.3.50. Unity Hub should prompt you to install the updated version next time you open the project. The Unity upgrade mainly contains engine bugfixes, so there are no expected changes in mod behavior.

1.0.1

- The following components have been merged into one component named DropLootAuthoring:
    - DropsLootAuthoring
    - DropsLootFromLootTableAuthoring
    - SeasonalLootAuthoring
    - DropsLootOnUseAuthoring
    - DropsLootWhenDamagedAuthoring
- HealthRegenerationAuthoring has been merged into HealthAuthoring.
- AnimationOrientationAuthoring has been merged into AnimationAuthoring.
- GrowingAuthoring options has been merged into SeedAuthoring and PlantAuthoring.
- LastAttackerAuthoring has been merged into EnemyAuthoring.

- Removed canBeHitAtSameTimeAsEnemy bool in MineableAuthoring as it was only used by PlayerGrave.
- Chests no longer need to be adjacent to the crafting station to pick materials from the chests but can be anywhere within a 10 tiles radius of the crafting station. The code that handles this has been reworked to function very differently from before.
- The ClientInput component isn't set outside of the prediction loop anymore. If you need to fetch or modify some input value outside of the prediction loop, fetch the data from ClientInputData. See updated InvertPlayerMovementSystem in SystemExample.

1.0

- Core Keeper is now available on consoles as well as PC. Modding is only available on PC, and we will most likely never support mods on consoles in the same way we do on PC. The platform owners (i.e. Sony, Microsoft and Nintendo) have strict requirements for all games that are released on their consoles, and there is no reasonable way to comply with them if players can modify the game to any meaningful extent.
- There is a known issue with mods not working properly on non-steam platforms due to some differences in the game initialization flow. We are working on a fix for this.
- Player controller has been changed to be server authoritative instead. This was also mentioned in the game changelog, but this is worth bringing up here as well as one of the biggest refactorings we've done. This means the player controller consists of a number of ECS systems. The code is still similar in places, but for example PlayerController have more static functions instead called from ECS.
- Apart from the player most enemies and objects are interpolated (shown with a server delay to avoid client-side computation). Some player interactions can still make them be predicted instead to improve responsiveness which means that a bunch of logic are now supporting prediction meaning a lot more systems are designed to run both on server and client.
- Completely new interaction system which is triggered from ECS. This part had to be completely remade instead of ported. Callbacks on EntityMonoBehaviours like OnUse should still be called in the same way via the new system.
- All SinglePugMap methods have been refactored to use world coordinates instead of render coordinates. Use EntityMonoBehaviour.ToWorldFromRender or Manager.camera.RenderOrigo to convert from render to world coordinates.
- RenderOrigo has been renamed to the more accurate RenderAnchor. You can access these using Manager.camera.GetRenderAnchor. If you don't need it for the full session, call Manager.camera.ReturnRenderAnchor to return it so that it can be cleaned up.
- PugSprite has been completely removed. This component was used to create MeshRenderers from SpriteRenderer to get the batching support from Unity, but has since been replaced mostly by using SpriteObject instead of SpriteRenderer.
- Even more enemies and objects have been converted from SpriteRenderer to using our own sprite rendering solution SpriteObject.
- More parts of the code is now using Burst, especially after the player controller rework. This is mainly done to improve performance. We are aware that

Burst-compiled code is a lot harder to patch, and we are already looking into ways to make this easier for modding.
- The new world generation uses a new GPU-driven framework to generate the terrain. This data is processed by an ECS system to create the ECS data that describes a world. The rules for this processing are described in the `TileTypeMapping` asset. You can also get direct access to the data produced by world generation shaders via the helper struct created by `Manager.worldgen.CreateProceduralDataRequester`. Each position in-game is described by a RGBA pixel. The current format is as follows:
    - Red channel: encodes the tile type. Use `CoreKeeperWorld.GetTileType` to get a more meaningful enum value.
    - Green channel: encodes the biome. Use `CoreKeeperWorld.GetBiome` to get a more meaningful enum value.
    - Blue channel: bit 0 (least significant) encodes if the position has only floor and no wall, bit 1 if there is a roof hole, and bit 3 if the Great Wall occupies this position. All other bits are currently unused.
    - Alpha channel: bits 0-2 encodes the ore type at this position. All other bits are currently unused.
- SerializeWorldSystem has been reworked to only load in things to the “live” world when the player is close enough (200 tiles). Previously, entities outside this range would be disabled but would still be kept in the live world. Some entities are always loaded like electricity, automation objects and bosses. The unloaded objects are kept in a separate ECS world (SerializeWorld) with a minimal representation. This world is what the world save file contains.
- AnimationAuthoring script was moved inadvertently, if you used this in any custom GameObject, you’ll have to reassign it. We try to not break references like that, but this one slipped the cracks in some refactor.
- WindUpAuthoring removed and same settings are moved to SecondaryUseAuthoring.
- Custom EntityMonoBehaviours aren’t updated as automatically as before, you might have to call some functions manually via GraphicalUpdate (see EnemyExample).
- Addressables are used in various places to load and unload assets to minimize memory footprint.
- A bunch of TimerSimple fields have been changed to use TickTimer. The typical reason is that this timer needs to be synchronized between server and client meaning that it needs to use NetworkTick instead of a regular floating point value to measure time.

This update contains lots of other small internal changes. Too many to list all of them here, but feel free to ask in the Discord and we’ll do our best to answer your questions.

0.7.5

- Updated Unity version to 2022.3.20f1. This is mostly in preparation for 1.0 and while it means there will be some changes we don’t expect anything noticeable but you will need the new version for the Mod SDK as well.

- For performance reasons, the update frequency of the ECS server world has been decreased from 30 to 20 updates per second. Any modded ECS code that uses the number of elapsed ticks as a metric for when to update will consequently run slower and may need to be adjusted. Similarly, MonoBehaviour.FixedUpdate is now called 30 times per second, down from 60 previously. Client world systems, objects implementing IGraphicalObject, and MonoBehaviour that only call the non-fixed update methods are unaffected, as they are all still called once per frame.
- All EntityUtility methods that previously accepted a ComponentType parameter have been refactored to instead take the relevant IComponentData type as a generic type parameter. Previously, each call to these functions resulted in a C# reflection lookup, which accumulated to have a significant performance impact.

0.7.4

- The mods that are loaded from mod.io will have an extra check that verifies the current version is supported by the mod, indicated by the Game Version tag. If this isn't set by the mod to the current version, it will still be possible to load the mod, but the player will get a warning first. This means it is more important to set this version tag correctly than before.
- The HealthChangeBuffer now has an entity field which indicates which entity is affected by the change and is stored in a singleton instead of on each entity.
- Expanded usage of the PugProperties system to avoid storing as much data on entities which instead can be shared. Each object entity now has an ObjectPropertiesCD component that can be used to look up shared data for that object. Added SetProperty functions to PugConverter where this data can be set.
- Some of the tile functions in Manager.multiMap have been replaced with Manager.multiMap.GetTileLayerLookup which gives you a job-compatible struct which can be used to access the same functions.

0.7.3

- You can choose a dedicated server install when choosing an install location in the SDK. Mods that are loaded via the SDK will also be detected properly by any connecting client even if the client uses the mod from mod.io. There is no automatic install of any missing mod that isn't installed via mod.io though.
- We are retiring the Translation component in favor of the LocalTransform component introduced in ECS 1.0. The main motivation for this change is to improve performance, as the Translation and LocalTransform must currently be kept in sync by copying the values back and forth. The Translation component will still be available in the project in order not to break serialized worlds, but no runtime entities are expected to have this component. This is one of the most commonly used components, but the following steps made the upgrade process fairly pain-free when we upgraded our own code:
    1. Use the following case-sensitive find-and-replace patterns. This took care of the vast majority of our usages.
        - `Translation translation` -> `LocalTransform transform`

        - `translation.Value` -> `transform.Position`
        - `, translation) in` -> `, transform) in`
        - `<Translation>` -> `<LocalTransform>`
        - `typeof(Translation)` -> `typeof(LocalTransform)`
    2. Fix all resulting compiler errors. The most common cause is that the patterns don't catch variable names in some cases.
    3. Replace all usages of `new Translation` with calls to `LocalTransform.FromPosition`.
    4. Find and replace any remaining usages of the Translation component.

  Unity's official ECS 1.0 upgrade guide might provide useful information on the new LocalTransform component: https://docs.unity3d.com/Packages/com.unity.entities@1.0/manual/upgrade-guide.html#update-transforms-in-your-project.
- Completely removed the Rotation component, as rotation of entities is not used anywhere in Core Keeper. Setting the Rotation field on LocalTransform also has no effect.
- EntityMonoBehaviour is the base class for all of our graphical objects, and over the years it has grown to an unmanageable size as it has to support all our objects' specific use cases. We are making a push to return to a more component-based approach. Our aim is to make it less daunting and error-prone to add new graphical objects to the game. For this update specifically, we have:
    1. Added support for multiple IGraphicalObject components on the same object. All such components **at the object root** are picked up and have their methods invoked as appropriate.
    2. Added separate interfaces for components that only provide spawn/despawn behavior. Prefer using these when possible to avoid the overhead of empty GraphicalUpdate methods being called each frame.
    3. Added several new components that control aspects of graphical objects that were previously managed with settings on EntityMonoBehaviour. We encourage you to when possible use these components instead of the corresponding EntityMonoBehaviour settings. The old settings have not been removed, but they will not be developed further and may be removed at a future time. These components are listed below, for more information see their in-editor tooltips and how they are used on existing assets. `SpriteSkinFromPaintColor`, `SpriteGradientMapFromPaintColor`, `SpriteVariationFromEntityDirection`, `SpriteVariationFromEntityVariation`, `SpriteSkinFromEntityAndSeason`, `EnableEffectsForEntityVariation`, `OffsetFromEntityDirectionOrVariation`
- The hierarchies of a lot of our graphical objects have also had an overhaul to improve performance. In most cases this means porting an older object to use our in-house sprite rendering solution (SpriteObject). Apart from this we have also done lots of small changes to increase performance in various ways, see the performance section of the 0.7.3 changelog for the Core Keeper application for more information on this.

0.7.2

- Upgraded Unity version to 2022.3.10f1.

- Upgraded DOTS/ECS to 1.0. See links for upgrade instructions. The Translation component should still work as before, and any baking changes aren't relevant because we are using our own baking system (PugConversion).
    - https://docs.unity3d.com/Packages/com.unity.entities@1.0/manual/upgrade-guide.html
    - https://docs.unity3d.com/Packages/com.unity.netcode@1.0/manual/upgrade-guide.html
    - https://docs.unity3d.com/Packages/com.unity.physics@1.0/manual/upgrade-guide.html
- Burst version upgraded to 1.8.9.
- A lot of ECS systems have been converted to use ISystem instead of SystemBase to improve performance.
- Added an option for you to bypass the security check, adding a notice about it to the user: "Caution: 'Elevated Access' mods have increased access to resources outside the game like user files and internet. For the best experience, install only from reputable sources." The reflection API might be removed in the future in favor of using this option instead.
- The Unity.Collections package is embedded in the SDK, but the only change is that we have added the old FixedString32/64 types which are still used for some serialized components. Changing these to the new versions would have broken serialization.
- The DOTS version uses a very different serialization format for ECS worlds. As a result there is a separate application in CoreKeeper_Data/StreamingAssets/Patcher which is used to convert saves from before 0.7.2 to an interim format that the new version can read.